

Quale DBMS?

Giuseppe Della Penna

Università degli Studi di L'Aquila

giuseppe.dellapenna@univaq.it

<http://people.disim.univaq.it/dellapenna>

Questo documento si basa sulle slide del corso di Laboratorio di Basi di Dati, riorganizzate per una migliore l'esperienza di lettura. Non è un libro di testo completo o un manuale tecnico, e deve essere utilizzato insieme a tutti gli altri materiali didattici del corso. Si prega di segnalare eventuali errori o omissioni all'autore.

Quest'opera è rilasciata con licenza CC BY-NC-SA 4.0. Per visualizzare una copia di questa licenza, visitate il sito <https://creativecommons.org/licenses/by-nc-sa/4.0>

DBMS Relazionali o NoSQL?

In questo corso studieremo i database *relazionali* (RDBMS). Tuttavia, negli ultimi anni sono sempre più diffusi e apprezzati i cosiddetti database *NoSQL*, che applicano altri modello di memorizzazione.

Quali sono le differenze tra questi tipi di DBMS?

Modelli di Memorizzazione

RDBMS

PostgreSQL, MySQL, Oracle, SQL Server,...

- Si basano sul **modello relazionale**.
- I dati sono organizzati in tabelle (relazioni) con schemi *rigidamente* definiti, colonne *tipizzate* e *vincoli* di integrità.
- L'accesso e la manipolazione dei dati avvengono tramite **SQL**, linguaggio dichiarativo standard.

NoSQL

Redis, MongoDB, Neo4j,...

- Rappresentano una famiglia eterogenea di sistemi progettati per *scalabilità orizzontale* (distribuzione dei dati tra più server paralleli), *flessibilità* dello schema e alte prestazioni su *grandi volumi* di dati.
- Adottano modelli dati diversi:
 - **key-value** (*Redis*),

- **document-oriented** (*MongoDB*),
- **graph** (*Neo4j*).

Punti di forza e debolezza

RDBMS

Punti di forza

- **Modellazione rigorosa**, ideale per domini ben strutturati.
- **Integrità** referenziale.
- **Consistenza** forte (ACID), garanzia di correttezza dei dati.
- Supporto **transazionale** avanzato: rollback, isolamento, locking.
- **SQL** standardizzato: elevata portabilità e maturità.
- **Query complesse**: join, aggregazioni, subquery efficienti.

Punti di debolezza

- Scarsa flessibilità dello schema, modifiche strutturali costose.
- Overhead di *normalizzazione*: più join, maggiore latenza.
- Scalabilità orizzontale complessa.
- Meno adatti a carichi altamente distribuiti.
- Prestazioni non ottimali su dati non strutturati (tipico dei *big data*).
- Difficoltà nell'adattarsi a dati eterogenei o semi-strutturati.

NoSQL

Punti di forza

- Elevata flessibilità dello schema.
- **Scalabilità orizzontale** nativa.
- Facilità di integrazione con **architetture distribuite** e cloud.
- Prestazioni elevate su **grandi volumi di dati**.
- Adatti a **dati semi-strutturati o non strutturati**.
- Ridotta latenza per **operazioni semplici e massive**.

Punti di debolezza

- Assenza di standard unificato.
- Consistenza spesso non immediatamente garantita (*BASE*).
- Supporto transazionale limitato o non uniforme.
- Rischio di duplicazione e incoerenza dei dati.

- Query complesse difficili o inefficienti.
- Join non supportati o demandati all'applicazione.

Criteri di scelta

1. Struttura dei dati

- Dati ben strutturati e stabili → RDBMS
- Dati eterogenei o in evoluzione → NoSQL

2. Requisiti di consistenza

- Necessità di consistenza forte (ACID) → RDBMS
- Accettabile consistenza eventuale (BASE) → NoSQL

3. Scalabilità

- Crescita moderata → RDBMS
- Crescita massiva e distribuita → NoSQL

4. Tipo di carico

- Query complesse e analitiche → RDBMS
- Accessi semplici e ad alta frequenza → NoSQL

5. Dominio applicativo

- Sistemi finanziari, ERP, gestionali → RDBMS
- Big data, IoT, social, logging → NoSQL

Confronto tra DBMS Relazionali

In questo corso useremo *MySQL community edition* come DBMS *relazionale* di riferimento per i nostri esempi e progetti. Tuttavia, rimanendo nell'ambito dei DBMS gratuiti, dobbiamo citare due alternative molto diffuse: *PostgreSQL* e *SQLite*.

Quali sono le differenze tra questi tre RDBMS?

Tecnologia

- MySQL è un database relazionale (*RDBMS*) puro.
- PostgreSQL è propriamente un Object-Relational DBMS (*ORDBMS*).
Per questo motivo, ad esempio, permette anche l'ereditarietà tra tabelle (anche se con supporto limitato).
- SQLite è un RDBMS puro *serverless*.

Tipi di dato

- PostgreSQL supporta tutti i tipi di dato MySQL, ma anche `BOOLEAN` e tipi avanzati come `RANGE` e `ARRAY`. Supporta inoltre il tipo `SERIAL` in sostituzione del flag `AUTO_INCREMENT` su `INTEGER` di

MySQL.

- In SQLite la tipizzazione è molto debole e dinamica. Dispone dei soli tipi `REAL`, `INTEGER`, `TEXT` (`VARCHAR` è un alias e la lunghezza è ignorata). Nessun tipo data/ora.

Per quel che riguarda il supporto JSON (tipico dei DBMS NoSQL)

- MySQL dispone del tipo `JSON` che memorizza i dati in binario e di numerose funzioni di supporto
- PostgreSQL supporta `JSON` (memorizzazione testuale) e `JSONB` (memorizzazione binaria).

Motore di storage

- In MySQL è selezionabile: ce ne sono diversi, come *MyISAM* o *InnoDB* (il default), con ottimizzazioni differenti.
- PostgreSQL e SQLite hanno un unico motore di storage.

Sistema transazionale

- PostgreSQL e SQLite hanno un sistema transazionale *completo*.
- MySQL non supporta le transazioni nel DDL.

Indici

- I tre DBMS supportano tutti gli indici *B-tree*, tuttavia MySQL e (soprattutto) PostgreSQL dispongono di altre tipologie di indice più avanzate.
- MySQL supporta le `FOREIGN KEY` solo con InnoDB.
- In SQLite le `FOREIGN KEY` sono presenti ma disabilitate di default.

Sintassi SQL

I dialetti dei tre DBMS coincidono su quasi tutto l'SQL più comune, ma con alcune estensioni o differenze notevoli. Ad esempio:

- In PostgreSQL si può usare `GENERATED AS IDENTITY` (SQL standard) come alternativa a `SERIAL` e quindi a `AUTO_INCREMENT` di MySQL.
- In PostgreSQL si può usare l'espressione `INSERT INTO ... VALUES (...) RETURNING id`,
- Per gestire i conflitti in inserimento
 - MySQL usa le sintassi `INSERT INTO ... VALUES ... ON DUPLICATE KEY UPDATE` (upsert), `INSERT IGNORE` e `REPLACE`.
 - PostgreSQL usa il costrutto più generale `INSERT ... ON CONFLICT (...) DO IGNORE / UPDATE`.
 - SQLite supporta anch'esso `INSERT ... ON CONFLICT FAIL / IGNORE / REPLACE` ma anche `REPLACE` e `INSERT OR REPLACE / IGNORE`.

Stored Procedure e Funzioni

- SQLite *non* supporta procedure e funzioni.
- MySQL ha il suo linguaggio procedurale interno.
- PostgreSQL permette di programmare con PL/pgSQL, PL/Tcl PL/Perl, PL/Python (PL/Python) e altri.

Il linguaggio è più esteso di quello di MySQL, ad esempio supporta `RETURNS TABLE` nelle funzioni.

Apprendimento

- MySQL è relativamente semplice da installare e utilizzare. Dispone di un ampio set di strumenti per utenti non esperti.
- PostgreSQL è più avanzato e complesso. Ha un set di strumenti limitato per utenti non esperti.
- SQLite non deve essere installato, ma inserito come libreria in programmi C, Java, Python, ecc., spesso integrandosi con il sistema di interfacciamento nativo del linguaggio per i DBMS (ad es. JDBC in Java). L'SQL semplificato lo rende più semplice da apprendere di MySQL.

Uso Tipico

- MySQL: principalmente *applicazioni web*.
- PostgreSQL: *applicazioni enterprise*, ma anche applicazioni web complesse.
- SQLite: memorizzazione dati strutturati per *piccole applicazioni, spesso desktop o mobili*.

Licenza

- La community edition di MySQL è *closed source* e rilasciata con licenza *GPL*. Le altre edizioni hanno licenze commerciali. MariaDB è un'alternativa open source in larga parte compatibile.
- PostgreSQL è *open source* con licenza *PostgreSQL License*, simile alla BSD o MIT.
- SQLite è *open source* e *Public Domain* (anche se le sue versioni per altri linguaggi/piattaforme potrebbero avere licenze leggermente più restrittive).